PROPOSAL · STARTING STAGE · ARCHITECTURE EDITION

Qlik-to-SQL Reconstruction: Architecture, Workflow and Technology

The starting-stage proposal with the full architecture drawn out, a step-by-step account of how the workflow runs, and the technology choices with the alternatives considered for each.

Document	Starting-stage proposal, architecture edition
Companion to	Starting-Stage Proposal and Starting-Stage Technical Design
Prepared for	Data & Engineering Leadership and technical reviewers
Date	October 2026
Status	Draft for review
Based on	Architecture v0.8 and the Qlik-first build plan v0.3

Contents

1. Summary
2. The starting point: Qlik on top of Dremio
3. Architecture at a glance
4. The full architecture
5. Components in depth
6. How the workflow runs
7. Worked example: one Qlik table to validated SQL
8. Inside the workflow: control, knowledge, AI, review
9. Validation and mismatch diagnosis
10. Re-runs and change handling
11. Agents in the starting stage
12. Technology stack and alternatives
13. Environments and deployment
14. Security and privacy controls
15. Delivery plan
16. Success measures
17. Risks and mitigations
18. Open questions that shape the build
19. Decision requested

1. Summary

The Agentic Dataset Context Engine (ADCE) starts with the capability that proves its central claim: **reconstructing how reported numbers are produced**. The starting stage reads Qlik scripts (QlikView and Qlik Sense), translates their logic into **Dremio SQL**, runs that SQL in Dremio, and checks the results against independent evidence: reload logs, QVD files and exported reports. Every result carries an explicit validation level.

This edition of the proposal explains the build in depth. It shows the complete architecture as one diagram, walks through every step of the workflow (what triggers it, what happens inside, what it records, where a person is involved and what happens when something goes wrong), follows a single Qlik table end to end, and lists every technology choice next to the alternatives that were considered.

Question	Short answer
What is built first?	Qlik script → BI model → Dremio SQL → validated result, with lineage, a review workbench and an exportable migration pack per application.
Is it a throw-away prototype?	No. The knowledge graph, review service, model gateway, Dremio connector and agent runtime are built once in their final shape; later stages add agents to them.
How is correctness shown?	A validation ladder (V0 parsed to V5 figures match). A result is never called correct without a level and its evidence.
Where does AI help?	Only where rules run out: unusual statements, complex expressions, SQL repair, diagnosing mismatches and reading exported reports. Every AI output is re-checked by code.
What does it touch?	Read-only access to Dremio and Qlik files. No writes to Dremio, no reloads or changes to Qlik, no row-level data sent to AI models.
Where does it run?	Locally first, then staging on Google Cloud; the same build is packaged for production and for installation in client environments.

Ask: approve the 12-week starting stage on 2–3 real Qlik applications, with read-only Dremio access, the Qlik artifacts listed in section 18, and a Google Cloud project for model access.

2. The starting point: Qlik on top of Dremio

The data behind the reports lives in Dremio. It is not structurally maintained: many near-identical views, redundant copies and dumps sit next to a few golden datasets. On top of it, queries and Dremio views shape the data, and Qlik load scripts read it, transform it further (joins, mappings, concatenations, calculated fields), sometimes pass it through QVD files between applications, and finally feed the charts, whose measures add another layer of logic.

Layer	Where the logic lives	What the starting stage does with it
Dremio datasets	Physical tables, copies, dumps, golden records	Catalogs them; checks keys and grain; flags when Qlik reads a copy and a golden source exists
Dremio views and queries	View SQL definitions; queries embedded in Qlik scripts	Parses view SQL into lineage; transpiles embedded SQL to the Dremio dialect
Qlik load scripts	LOAD, SQL SELECT, JOIN, KEEP, CONCATENATE, ApplyMap, variables, loops	Parses into a tool-neutral BI model; translates each statement into SQL

Layer	Where the logic lives	What the starting stage does with it
QVD files	Intermediate tables passed between Qlik applications	Builds QVD chains so the final SQL reads only Dremio datasets; uses QVD headers as evidence
Qlik data model and charts	Associations between tables; measures with set analysis, TOTAL and Aggr	Translates measures through a shared expression form; composes SQL per visual; checks grain

Read-only towards Dremio and Qlik · no row-level data sent to AI models · every result labelled with its evidence level

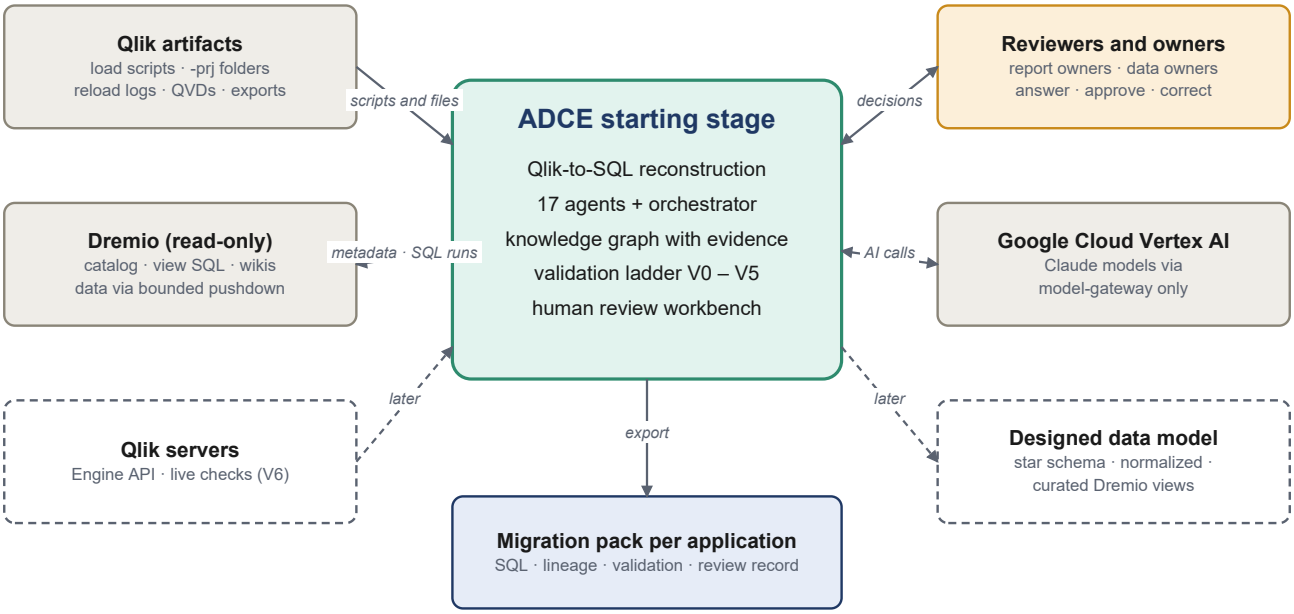


Figure 1. The starting stage in its environment. Dashed items are later stages.

3. Architecture at a glance

The architecture has seven layers. Each layer has one job and talks to its neighbours through a small, versioned contract, so parts can be built, tested and replaced independently.

Layer	Job	Starting-stage components
Interfaces	People and files come in; packs go out	Review workbench, API gateway, Keycloak
Control plane	Decide what runs next; manage human review; govern every AI call	orchestrator (LangGraph), hitl-service, model-gateway
Agent workers	Do the work: parse, resolve, translate, compose, check, reconcile, publish	17 agents in four worker services
Knowledge access	The only way knowledge is written or read; enforces provenance and trust	kg-mcp over kg_port (the evolved Brahmastra engine)
Storage	Keep facts, history, files, traces	Postgres, Neo4j, MinIO/GCS, DuckDB, Langfuse
External systems	Where data and models live	Dremio (read-only), Vertex AI; Qlik servers later
Shared library	One definition of every contract	adce-core: AgentSpec, BI IR, ExprAST, schemas, ports

Design rules that shape every component

1. **Deterministic first.** Parsing, transpiling, resolution and checks are code. AI handles the long tail, and its output is re-parsed and re-checked by the same code.
2. **Knowledge has one writer.** Agents propose facts with evidence; only kg-mcp writes, after validation and policy checks. The append-only fact log is the source of truth; the graph is a projection that can be rebuilt.
3. **Nothing silently dropped or assumed.** Unparsed statements, unresolved variables and uncertain mappings become flagged items with a reason, never guesses.
4. **Evidence for every claim.** Every fact links to the script span, catalog entry, log line, QVD header or query result that supports it.
5. **Read-only and bounded.** No writes to Dremio or Qlik; every query is bounded, labelled and rate-limited.
6. **Reproducible.** The same inputs, software and prompt versions regenerate the same migration pack (memoized AI calls, versioned prompts).
7. **Human review is data, not a blocking call.** The pipeline keeps going; decisions re-run only what they affect.
8. **Built to ship.** The same images and chart run locally, in staging, in production and inside a client's environment.

4. The full architecture

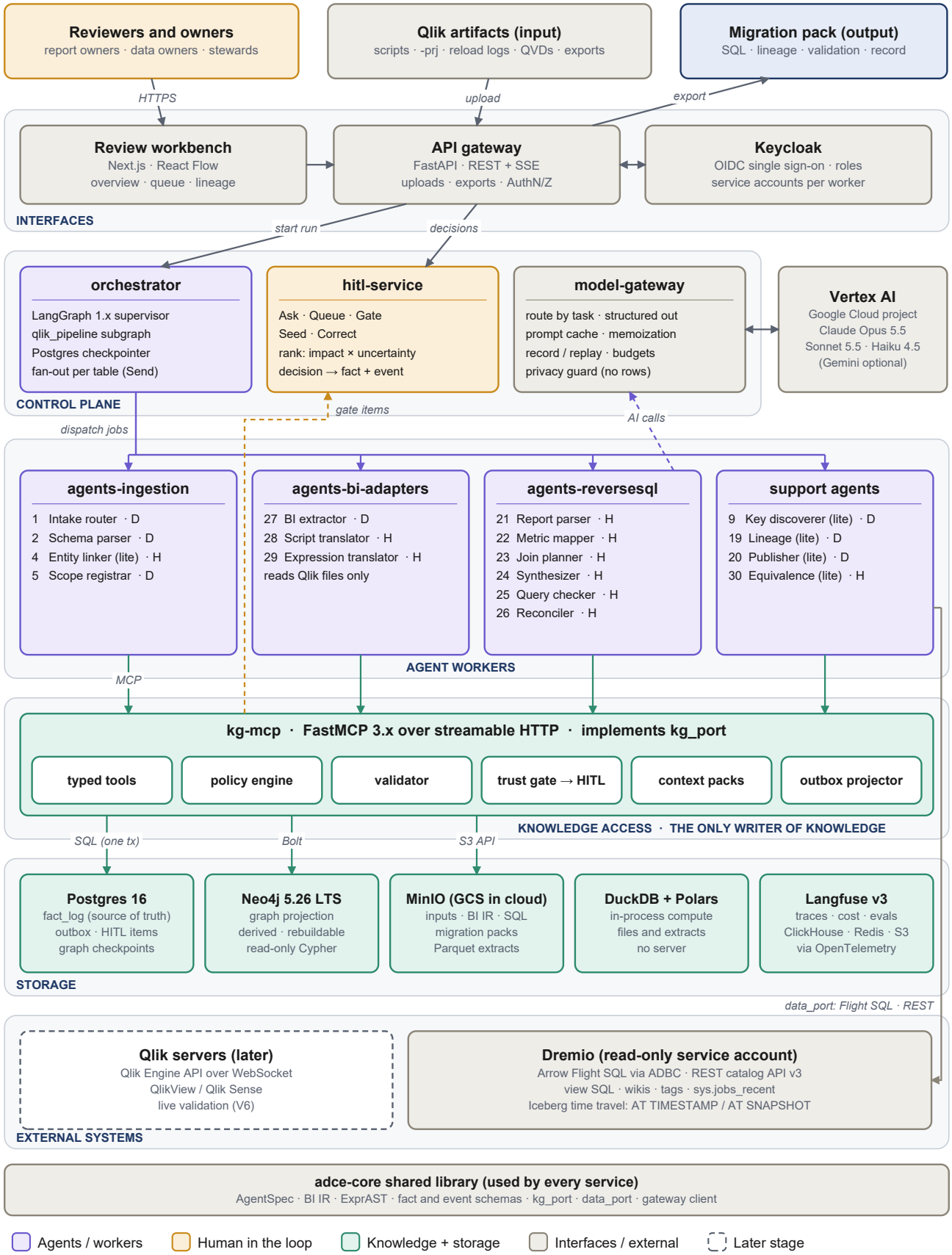


Figure 2. The full starting-stage architecture. Protocols are written on the connections. Agents call the model gateway for AI, data_port for Dremio and kg-mcp for knowledge; nothing else writes knowledge.

Figure 2 shows every service, store and connection in the starting stage. Read it top to bottom; the numbered notes below follow the same order.

1. **Inputs and outputs.** Reviewers use the workbench in a browser. Qlik artifacts are uploaded through the API gateway (scripts, QlikView project folders, reload logs, QVD files, exported reports). Migration packs are exported through the same gateway.
2. **Interfaces.** The API gateway is the single entry point. It validates Keycloak tokens, streams uploads into object storage with content hashes, starts runs and serves review data and progress (server-sent events).
3. **Control plane.** The orchestrator runs the qlik_pipeline graph per application and fans out per Qlik table. hitl-service holds review items and turns decisions into facts and events. model-gateway is the only route to AI models on Vertex AI.
4. **Agent workers.** Four worker services host the 17 agents. All agents share one runtime contract: receive a job, read a context pack, do the work, propose facts with evidence, emit a trace. D marks deterministic agents and H marks hybrid ones (rules first, AI for the rest).
5. **Knowledge access.** kg-mcp exposes typed tools over MCP. Every write passes the validator and policy engine, lands in the fact log in one transaction with an outbox row, and is projected into the graph. Facts that need review go through the trust gate to hitl-service.
6. **Storage.** Postgres holds the fact log, outbox, review items and orchestration checkpoints. Neo4j holds the derived graph. MinIO (GCS in the cloud) holds files, BI IR, SQL and packs. DuckDB and Polars compute on files in process. Langfuse stores traces, cost and evaluations.
7. **External systems.** Dremio is reached only through data_port with a read-only service account: Arrow Flight SQL for queries and the REST catalog API for metadata and job history. Qlik servers are a later stage.
8. **Shared library.** adce-core defines every contract once (Pydantic models with JSON Schema), so services cannot drift apart.

Connections and protocols

From → to	Protocol	Authentication	Notes
Browser → workbench / API gateway	HTTPS, REST + SSE	OIDC (Keycloak), PKCE	Roles: viewer, reviewer, approver, admin
API gateway → orchestrator, hitl-service	Internal HTTP	Service tokens	Start runs, submit decisions
Orchestrator → workers	Jobs from the Postgres outbox (LISTEN/NOTIFY)	Service accounts	Redis Streams replaces this in later stages
Workers → kg-mcp	MCP, streamable HTTP	Client-credentials token per worker	Typed tools only; no direct database access
Workers → model-gateway	Internal HTTP	Service token	Route name + inputs + output schema
model-gateway → Vertex AI	HTTPS (Anthropic Vertex SDK)	Google service account	Approved project and region
Workers → Dremio (data_port)	Arrow Flight SQL (ADBC); REST catalog API v3	Read-only Dremio service account (token)	Bounded, labelled queries
kg-mcp → Postgres / Neo4j	SQL / Bolt	Database credentials from secrets	Writes only from kg-mcp and the projector

From → to	Protocol	Authentication	Notes
Workers, gateway → MinIO / GCS	S3 API	Scoped keys / workload identity	Content-addressed objects
All services → Langfuse	OpenTelemetry / Langfuse SDK	Project keys	Prompt version, model, tokens, cost, latency

5. Components in depth

5.1 API gateway

Field	Detail
Responsibility	Single entry point for people and files; authentication and authorization; uploads, exports, run control and review data.
Inside	FastAPI routers for applications, uploads, runs, review items, lineage and exports. Uploads stream to object storage and are keyed by SHA-256, so a re-upload is recognized. Progress is pushed with server-sent events.
Interfaces	REST + SSE to the workbench; internal calls to orchestrator, hitl-service and kg-mcp read tools.
State	Stateless; everything lives in the stores.
Scaling and failure	Horizontal. Requests are idempotent by content hash and run ID, so retries are safe.

5.2 Review workbench

Field	Detail
Responsibility	Where people answer questions, review mappings and approve SQL.
Inside	Next.js and React. Three screens: application overview (inputs, coverage, level per table and figure, open items), review queue (Ask, Queue and Gate items with batch actions) and lineage view (React Flow graph from Dremio datasets to visuals, from week 6).
Interfaces	API gateway only; never talks to stores directly.
State	None beyond the browser session.
Scaling and failure	Static assets and server rendering; a decision is saved before the UI moves on.

5.3 Keycloak

Field	Detail
Responsibility	Identity and roles for people and services.
Inside	One realm per environment; roles viewer, reviewer, approver and admin; a client-credentials service account per worker service; federation to a client's identity provider (OIDC or SAML) in client installs.
Interfaces	OIDC to the workbench and API gateway; token validation in every service.
State	Its own database (Postgres).
Scaling and failure	Standard Keycloak clustering in production; tokens are short-lived.

5.4 Orchestrator

Field	Detail
Responsibility	Runs the qlik_pipeline per application: decides what runs next, fans out per table, handles retries, pauses and re-entry.
Inside	LangGraph 1.x StateGraph. The state carries the application, run, table list and per-table status. Nodes dispatch agent jobs. Send fans out per Qlik table. The Postgres checkpointer saves state after every node. interrupt() is used only for Ask items; Queue and Gate items never block.
Interfaces	Consumes events (upload.classified, fact.confirmed, dataset.changed); writes jobs to the outbox; reads facts through kg-mcp.
State	Checkpoints in Postgres; no in-memory state that matters.
Scaling and failure	Concurrency limits per application and per Dremio. A crash resumes from the last checkpoint. Jobs are idempotent because fact IDs are stable hashes of their content and scope.

5.5 hitl-service

Field	Detail
Responsibility	Holds every review item, ranks them, records decisions and turns them into facts and events.
Inside	Five modes: Ask (an agent cannot continue), Queue (non-blocking review), Gate (approval before trust is raised), Seed (known values given up front, such as connection mappings) and Correct (a reviewer edits a result). The queue is ordered by impact (facts that depend on the item) times uncertainty. Decisions are stored with the original proposal and reused as examples.
Interfaces	Receives gate items from kg-mcp; serves the workbench through the API gateway; writes decision facts through kg-mcp; emits fact.confirmed or fact.rejected.
State	Review items and decisions in Postgres.
Scaling and failure	Stateless service over Postgres; a decision is committed once, then emitted.

5.6 model-gateway

Field	Detail
Responsibility	The only route to AI models; makes AI calls safe, reproducible, affordable and observable.
Inside	Named routes (for example translate_statement, translate_expression, repair_sql, diagnose_mismatch, parse_export) each fix the model, effort, prompt version and output schema. Steps: privacy guard, memo lookup, budget check, prompt assembly with caching of the stable prefix, call, schema validation, record and trace (Figure 7).
Interfaces	Internal HTTP from agents; Vertex AI through the Anthropic Vertex SDK; Gemini reachable through the same adapter interface.
State	Memo and replay store (Postgres and object storage); budgets per task and run.
Scaling and failure	Horizontal. Backoff on rate limits and server errors; client-side fallback on refusals; one schema retry, then the item goes to the review queue.

5.7 kg-mcp (knowledge service)

Field	Detail
Responsibility	The single writer of knowledge; serves context to agents; enforces provenance and trust.

Field	Detail
Inside	Typed tools (register_entity, add_context, propose_fact, get_entity, get_context_pack, search_context, get_source_context, read-only query). Validator: schema, scope and referential checks. Policy engine: which agent may write which fact kinds and the highest trust it may assign (agents write at most inferred or validated; only people confirm). Trust gate: sends facts to review by confidence × impact. Context assembler: builds token-budgeted packs. Outbox projector: keeps Neo4j in step with the fact log.
Interfaces	MCP over streamable HTTP (FastMCP 3.x), implemented on kg_port, the boundary to the knowledge engine (Brahmastra evolved in place).
State	Fact log and outbox in Postgres; projection in Neo4j.
Scaling and failure	Horizontal for reads; writes are short transactions. The projection can be rebuilt from the fact log at any time.

5.8 Agent workers

Field	Detail
Responsibility	Host the agents in four services: agents-ingestion, agents-bi-adapters, agents-reversesql and support agents.
Inside	Each agent declares an AgentSpec: name, version, kind, triggers, tools it may call, routes it may use, and its output fact kinds. The runtime loop is: receive job → get context pack → work (code, AI or both) → propose facts with evidence → emit trace.
Interfaces	Jobs from the outbox; kg-mcp tools; model-gateway; data_port; object storage.
State	None; inputs and outputs live in the stores.
Scaling and failure	Each service scales on queue depth. agents-bi-adapters is separate because it will later hold Qlik server credentials and long-lived sessions.

5.9 data_port (Dremio connector)

Field	Detail
Responsibility	Every interaction with Dremio, under strict read-only and load rules.
Inside	Operations: list catalog, get dataset (schema, view SQL, wiki, tags), run bounded query, explain, profile key uniqueness, query job history, and point-in-time query (AT TIMESTAMP or AT SNAPSHOT on Iceberg tables). A statement-type check (sqlglot) rejects anything that is not a SELECT. Queries carry a label, a row limit, a timeout and a concurrency slot; results are cached by SQL hash and snapshot.
Interfaces	ADBC Flight SQL driver for queries; REST catalog API v3 for metadata; sys.jobs_recent for query history.
State	Result cache only.
Scaling and failure	Global concurrency limit and off-peak scheduling for heavy checks; retries with backoff; a failing query becomes a recorded result, not a crash.

6. How the workflow runs

Figure 3 shows the eleven steps for one Qlik application. Agent steps run down the middle; inputs and Dremio feed in from the left; every step records knowledge; review items go to people; outputs leave on the right. Steps 4 to 9 run per Qlik table, in parallel.

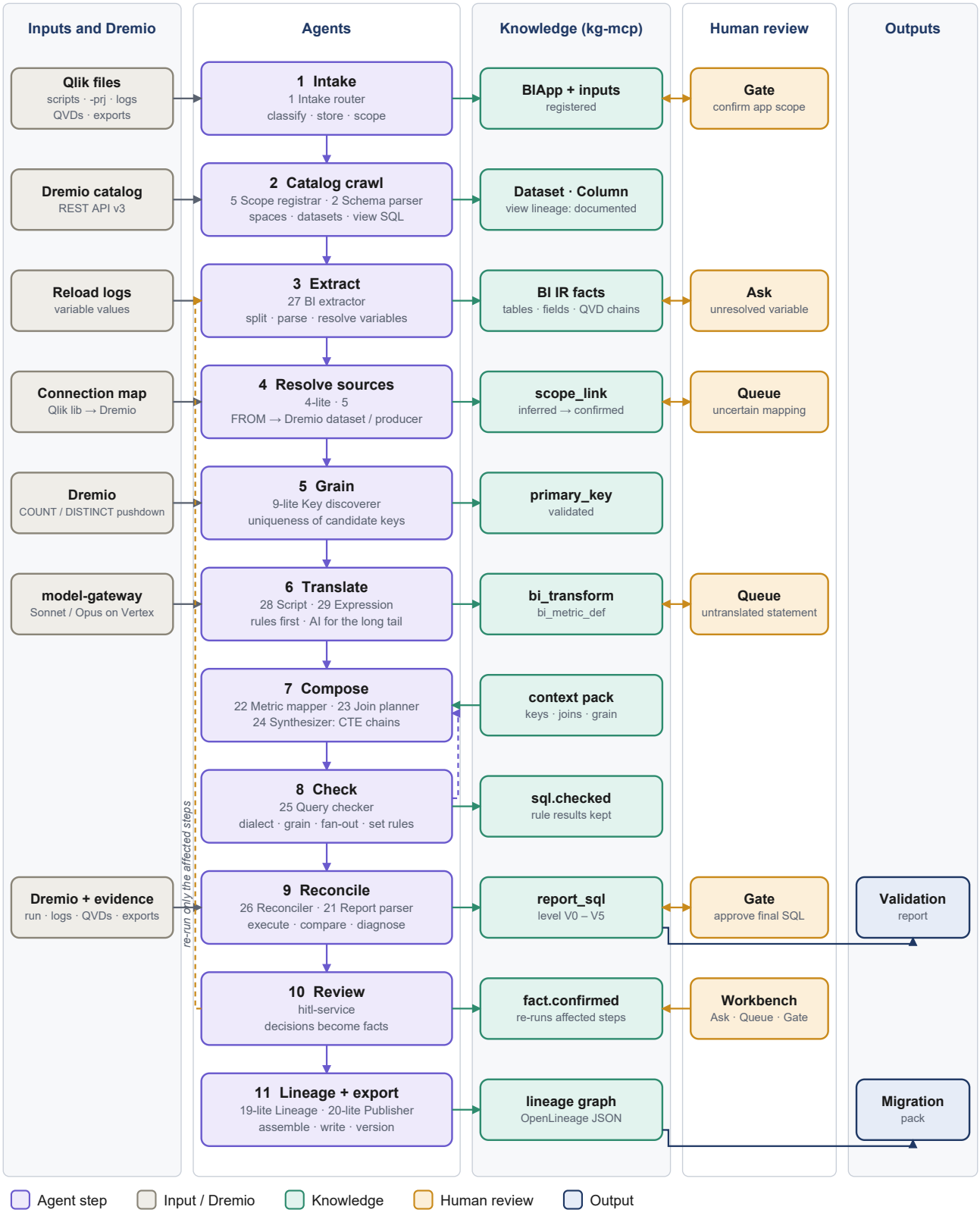


Figure 3. The starting-stage workflow as swim lanes. Dashed lines: the repair loop (check back to compose) and re-runs triggered by review decisions.

Step by step

Step 1. Intake

Field	Detail
Trigger	Files uploaded for an application, or new files for an application already registered.
Inputs	Load scripts (.qvs or text), QlikView project folders (-prj), Qlik Sense app files where readable offline, reload logs, QVD files, exported reports, connection list.
Inside	Intake router (1) stores each file in object storage by content hash, classifies it (script, project folder, app file, reload log, QVD, export, connection map) from structure and extension, and groups files under the application's scope.
Records	BIApp entity and its inputs; event upload.classified[bi_app].
People	Gate: confirm the application scope and which files belong to it.
If it fails	Unknown file types are listed as unclassified, never dropped. Duplicate uploads are recognized by hash.

Step 2. Catalog crawl

Field	Detail
Trigger	First run for a Dremio source, or dataset.changed from a scheduled crawl.
Inputs	Dremio REST catalog API: spaces, sources, folders, datasets, columns, view SQL, wikis, tags.
Inside	Scope registrar (5) registers spaces and datasets. Schema parser (2) records columns and types and parses every view's SQL with sqlglot into column-level lineage between datasets.
Records	Dataset and Column entities; view lineage with documented trust; event dataset.registered.
People	None in this step.
If it fails	Views whose SQL does not parse are flagged with the parser message; the crawl continues.

Step 3. Extract

Field	Detail
Trigger	upload.classified[bi_app].
Inputs	Scripts, project folders, reload logs, QVD headers.
Inside	BI extractor (27) splits scripts into statements (aware of comments, strings and brackets), parses them with the Qlik grammar, inlines include files with their source spans, resolves variables (LET/SET, then values printed in reload logs), unrolls simple loops, reads QVD header XML (creator document, record count, lineage) and records every STORE and LOAD FROM .qvd.
Records	The BI IR (tables, fields, sources, transforms, keys, variables, measures) in object storage; BI model facts; QVD chain links; coverage per application; event bi_ir.ready.
People	Ask: a variable that cannot be resolved from the script or the logs.
If it fails	A statement the grammar does not cover becomes kind: other with its raw text and span; it is counted in coverage and sent to the translator's AI route or to review.

Step 4. Resolve sources

Field	Detail
Trigger	bi_ir.ready, or a confirmed connection mapping.
Inputs	Each FROM target in the BI IR; the Qlik connection list; the Dremio catalog.
Inside	Each source is resolved in order: an explicit Dremio path in embedded SQL; the reviewed connection mapping (Qlik library → Dremio source); a QVD produced by another known application (follow the chain); a name and column-signature match in the catalog. Copies are detected by Equivalence (30-lite) so the exact copy Qlik reads is used, and a golden alternative is noted.
Records	scope_link facts with trust inferred (confirmed after review); missing producers named.
People	Queue: uncertain mappings, with the candidates and their evidence side by side.
If it fails	An unresolved source marks its tables not resolved; the rest of the application continues.

Step 5. Grain

Field	Detail
Trigger	Sources resolved for a table.
Inputs	Candidate keys from the BI IR (join fields, fields named like keys) and the Dremio datasets.
Inside	Key discoverer (9-lite) runs bounded COUNT and COUNT DISTINCT queries in Dremio to test uniqueness of candidate keys for every dataset Qlik reads.
Records	primary_key facts with trust validated, or a recorded duplicate-key finding.
People	None; findings feed the checker.
If it fails	Heavy checks are deferred to the off-peak window; the table continues with the grain marked unknown, which the checker treats as a risk.

Step 6. Translate

Field	Detail
Trigger	bi_ir.ready for a table (fan-out per table).
Inputs	The table's statements and measures; schemas of its inputs; confirmed translations of similar statements.
Inside	Script translator (28): embedded SQL transpiled to Dremio with sqlglot; Qlik transforms mapped by the rule table (LOAD → SELECT, preceding LOAD → nested CTE, JOIN → explicit join on all common fields, KEEP → semi-join, CONCATENATE → UNION ALL with aligned columns, ApplyMap → LEFT JOIN with default, WHERE/IF/date functions → SQL equivalents). Expression translator (29): measures parsed into ExprAST, then rendered as SQL (set analysis → filtered aggregates, TOTAL and Aggr → window functions or sub-aggregates). Statements outside the rules go to Sonnet 5.5, harder ones to Opus 5.5; every AI output is re-parsed and rejected if invalid.
Records	bi_transform and bi_metric_def facts with the rule or route used; events bi_transform.ready, bi_metric_def.ready.
People	Queue: statements that could not be translated, with the reason.
If it fails	Untranslated statements are listed in the pack; the table's level stops at V1.

Step 7. Compose

Field	Detail
Trigger	Translations ready for a table or visual.
Inputs	Translated fragments; keys and grain; joins from the Qlik data model and Dremio views.
Inside	Metric mapper (22) maps measures to fields. Join planner (23) plans joins from the Qlik associations, checked against confirmed keys. Synthesizer (24) assembles one CTE per statement in script order, so every line of SQL traces back to a script span, and composes SQL per visual.
Records	sql.candidates with the CTE-to-statement map.
People	None in this step.
If it fails	A join that cannot be planned is flagged as a conflict for review.

Step 8. Check

Field	Detail
Trigger	sql.candidates.
Inputs	Candidate SQL; knowledge about keys, grain and joins.
Inside	Query checker (25): Dremio dialect validity (sqlglot plus Dremio EXPLAIN); grain-aware aggregation (measures aggregated at their own table grain before joining, or the query is rejected for fan-out); set-analysis override rules; TOTAL and Aggr scope; NULL and dual-value behaviour; DISTINCT across associated tables. Failures go to a repair route (Opus 5.5) at most twice.
Records	sql.checked with every rule result; the table reaches V1.
People	None; repeated failures go to the queue.
If it fails	After two repairs the SQL is kept with its failed rules and queued; it is never presented as valid.

Step 9. Reconcile

Field	Detail
Trigger	sql.checked.
Inputs	Checked SQL; reload logs; QVD headers and contents; exported reports (parsed by Report parser 21).
Inside	Reconciler (26) runs the SQL in Dremio through data_port, as of the reload time where time travel is available, and compares: row counts per table (V3), column totals, distinct counts and ranges (V4), and figures per visual (V5). Differences are diagnosed (section 9).
Records	report_sql with the level reached, the comparisons, tolerances and their reasons.
People	Gate: approval of the final SQL by the report owner.
If it fails	A mismatch is routed back to the step that most likely caused it (re-plan, re-translate, re-resolve) or to review; the level reached so far is kept.

Step 10. Review

Field	Detail
Trigger	Any review item; reviewers work at their own pace.
Inputs	Ask, Queue and Gate items with proposals and evidence.
Inside	hitl-service records the decision, kg-mcp appends it as a fact (confirmed, rejected or corrected; the old version is superseded, never deleted) and emits fact.confirmed or fact.rejected.
Records	Decision facts; examples for future translations and mappings.
People	Report owners and data owners in the workbench.
If it fails	Undecided items stay open and are listed in the pack.

Step 11. Lineage and export

Field	Detail
Trigger	All tables of an application have a final level, or an export is requested.
Inputs	Facts for the application.
Inside	Lineage (19-lite) assembles the path from Dremio datasets through views, Qlik tables and QVDs to visuals. Publisher (20-lite) writes the migration pack: SQL, lineage (Markdown and OpenLineage JSON), validation report, review record and a manifest of inputs and versions.
Records	A versioned pack in object storage; event pack.published.
People	Owners receive the pack.
If it fails	Packs are regenerated from facts; nothing is lost.

7. Worked example: one Qlik table to validated SQL

A simplified but realistic script shows what each step produces. Names and numbers are illustrative.

7.1 The input: a Qlik load script

```
LIB CONNECT TO 'Dremio_Sales';
LET vStartDate = '2025-01-01';

Customers:
LOAD CustomerID, Upper(Name) AS CustomerName, Region;
SQL SELECT CustomerID, Name, Region FROM "sales"."crm"."customers";

Orders:
LOAD OrderID, CustomerID, Date(OrderDate) AS OrderDate, Amount,
     If(Amount > 1000, 'Large', 'Small') AS OrderSize
WHERE Status <> 'Cancelled';
SQL SELECT OrderID, CustomerID, OrderDate, Amount, Status
FROM "sales"."erp"."orders_v2" WHERE OrderDate >= '$(vStartDate)';

LEFT JOIN (Orders) LOAD CustomerID, Region RESIDENT Customers;
DROP TABLE Customers;
STORE Orders INTO [lib://QVD/Orders.qvd] (qvd);
```

Chart measure, *Revenue per customer from large orders*, by Region: `Sum({<OrderSize={'Large'}>} Amount) / Count(DISTINCT CustomerID)`

7.2 What the extractor records (BI IR excerpt)

```
table Orders  (span: lines 8-13, 15; stored at line 17)
  source   : sql  "sales"."erp"."orders_v2"      connection: Dremio_Sales
  variables: vStartDate = '2025-01-01'           (resolved from LET, confirmed by reload log)
  transform: preceding_load  fields[OrderID, CustomerID, OrderDate=Date(..), Amount, OrderSize=If(..)]
             where  Status <> 'Cancelled'
  transform: left_join  from Customers  on [CustomerID]   (common fields)  adds [Region]
  store    : lib://QVD/Orders.qvd                -> QVD chain link for consuming apps
table Customers (lines 4-6; dropped at line 16, used only through the join)
measure  revenue_per_large_customer = Sum({<OrderSize={'Large'}>} Amount) / Count(DISTINCT CustomerID)
coverage : 9 of 9 statements parsed
```

7.3 What resolution and grain checks add

- Dremio_Sales is mapped to the Dremio source *sales* (reviewed connection mapping), so both FROM targets resolve to datasets with documented trust.
- Equivalence (30-lite) finds two near-identical copies of orders_v2 (orders_v2_bkp and a dated dump). The reconstruction uses orders_v2, exactly what Qlik reads, and notes the copies in the pack.
- CustomerID is unique in crm.customers (validated), so the LEFT JOIN cannot multiply Orders rows. If it were not unique, Qlik would multiply rows too; the SQL would reproduce that faithfully and the duplicate keys would be reported as a data issue.

7.4 What translation and composition produce

```
-- Qlik table: Orders   (app: Sales Overview; each CTE maps to a script statement)
WITH customers AS (                                  -- lines 4-6
  SELECT CustomerID, UPPER(Name) AS CustomerName, Region
  FROM "sales"."crm"."customers"
),
orders_src AS (                                      -- lines 12-13, vStartDate resolved
  SELECT OrderID, CustomerID, OrderDate, Amount, Status
  FROM "sales"."erp"."orders_v2"
  WHERE OrderDate >= '2025-01-01'
),
orders AS (                                          -- lines 8-11 (preceding LOAD)
  SELECT OrderID, CustomerID, CAST(OrderDate AS DATE) AS OrderDate, Amount,
         CASE WHEN Amount > 1000 THEN 'Large' ELSE 'Small' END AS OrderSize
  FROM orders_src
  WHERE Status <> 'Cancelled'
),
orders_final AS (                                    -- line 15 (LEFT JOIN on common field CustomerID)
  SELECT o.*, c.Region
  FROM orders o
  LEFT JOIN (SELECT CustomerID, Region FROM customers) c ON o.CustomerID = c.CustomerID
)
SELECT * FROM orders_final;

-- Measure by Region (set analysis becomes a filtered aggregate; Qlik division by zero gives NULL)
SELECT Region,
       SUM(CASE WHEN OrderSize = 'Large' THEN Amount END)
         / NULLIF(COUNT(DISTINCT CustomerID), 0) AS revenue_per_large_customer
FROM orders_final
GROUP BY Region;
```

7.5 How it is validated

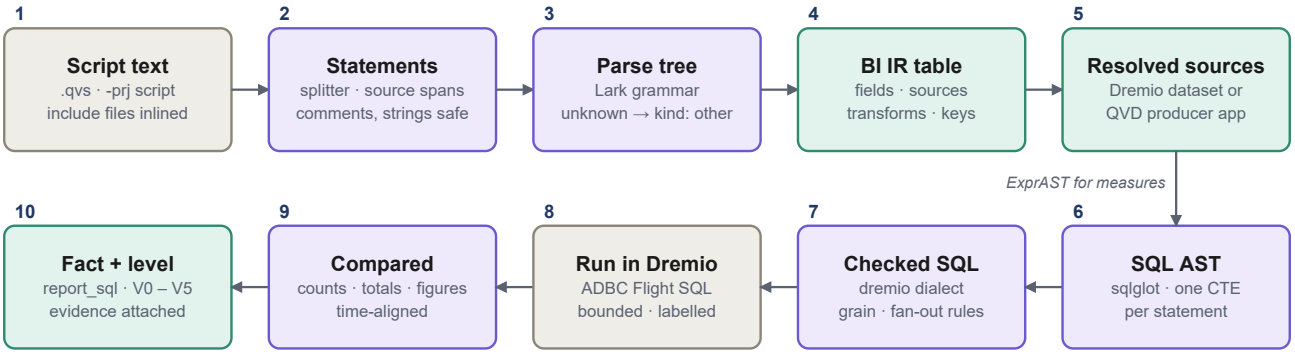
Level	Evidence	Result (illustrative)
V0 Parsed	9 of 9 statements parsed	Pass
V1 Checked	Dremio EXPLAIN succeeds; grain rule passes (CustomerID unique); no fan-out	Pass
V2 Executes	Query runs in Dremio with the expected columns	Pass

Level	Evidence	Result (illustrative)
V3 Row counts	Reload log: 48,213 lines fetched from orders_v2 = orders_src count; QVD header NoOfRecords 47,902 = orders_final count (queried AT TIMESTAMP of the reload)	Pass, exact
V4 Data	SUM(Amount), COUNT DISTINCT CustomerID, MIN/MAX OrderDate equal to the Orders.qvd contents	Pass
V5 Figures	Exported chart: 4 regions, values match to 0.01	Pass

The pack for this application then contains the SQL above, the lineage from orders_v2 and customers through Orders.qvd to the chart, the validation table, the note about the two copies of orders_v2, and the reviewer's approval.

8. Inside the workflow: control, knowledge, AI, review

8.1 Data flow for one table



Numbers show the order. Every box is stored: inputs and BI IR in MinIO, results as facts in the knowledge graph.

Figure 4. The ten artifacts one Qlik table passes through. Each is stored and can be inspected.

8.2 Orchestration

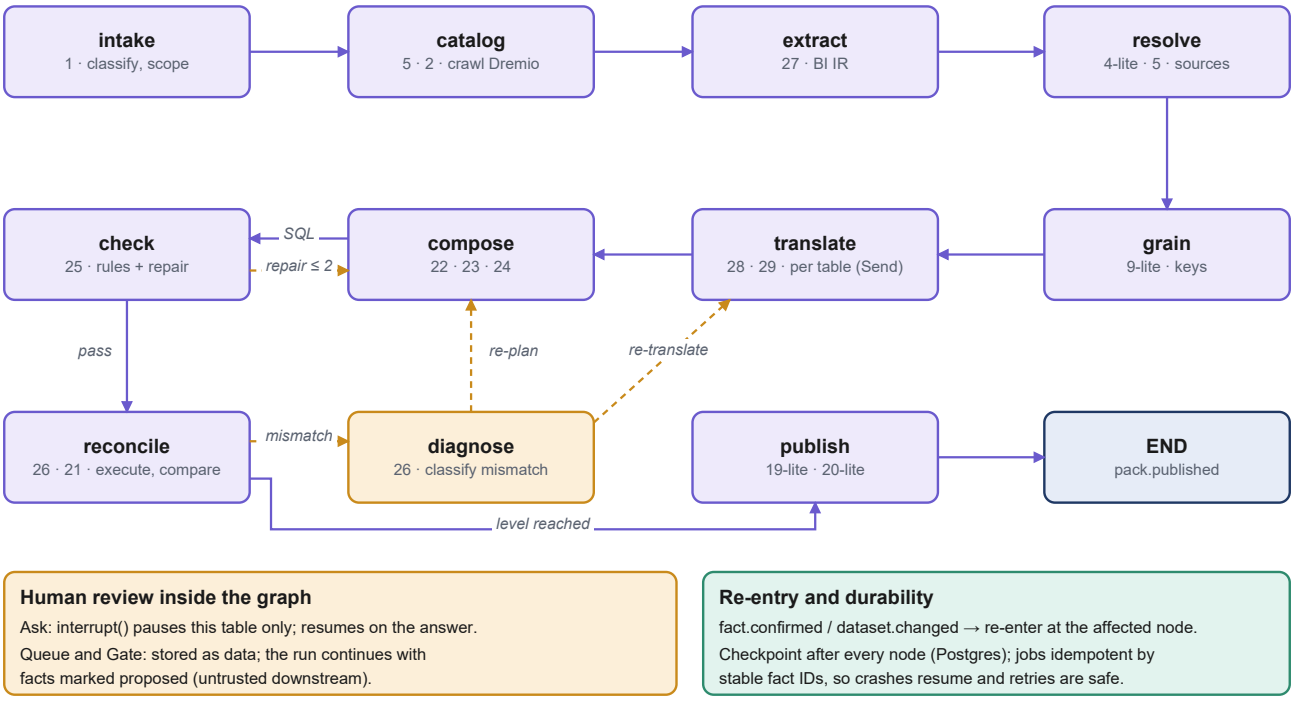


Figure 5. The qlik_pipeline graph. Solid: normal path. Dashed: repair and diagnosis loops.

- **One graph per application, fan-out per table.** After resolve and grain, Send creates one branch per Qlik table, so tables translate, check and reconcile in parallel within concurrency limits.
- **Checkpoint after every node.** State is saved in Postgres; a restarted orchestrator continues where it stopped.
- **Bounded loops.** check → compose repair runs at most twice; reconcile → diagnose routes to the most likely cause once per new evidence, so runs always finish.
- **Re-entry instead of restarts.** fact.confirmed, dataset.changed and new inputs re-enter the graph at the affected node for the affected tables only (section 10).

8.3 How knowledge is written

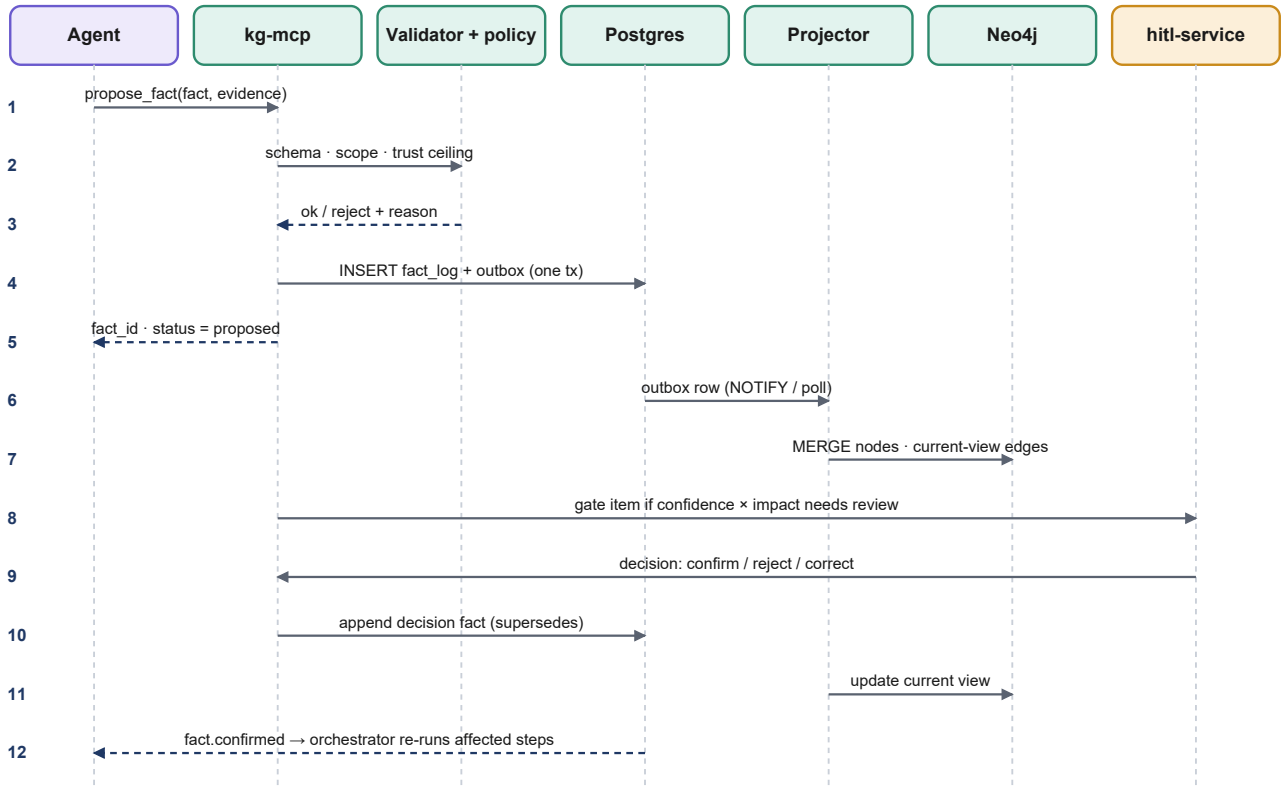
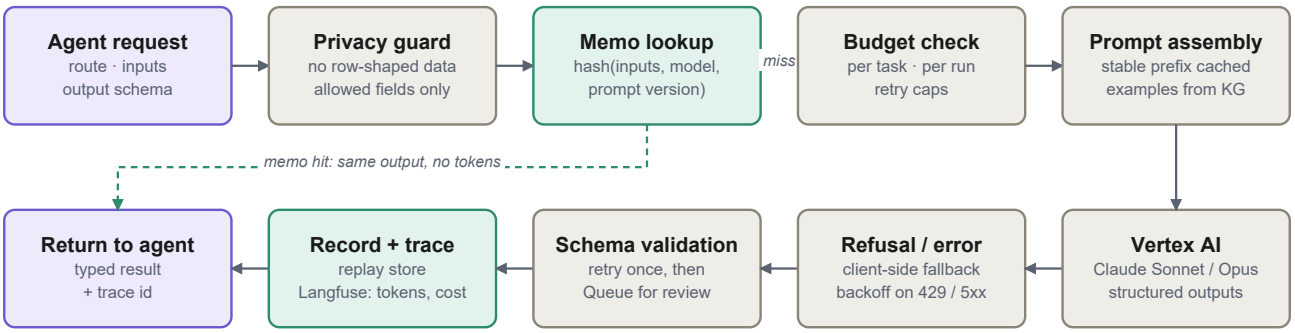


Figure 6. The write path. Agents never write to a store directly.

- **Facts are reified.** A fact (for example, "orders_v2.CustomerID is a primary key") is a record with kind, subject, value, trust, confidence, evidence links, the agent and version that proposed it, and two time ranges: when it is true and when it was recorded.
- **Append-only.** Corrections and decisions are new facts that supersede old ones. History is never lost, and any past state can be reconstructed.
- **Trust levels.** confirmed (a person decided), documented (read from a source such as a script or view definition), validated (checked against data), inferred (proposed by reasoning). Agents cannot assign confirmed.
- **Outbox.** Each fact and its event are written in one transaction, so the graph and every consumer see exactly what was committed.

8.4 How an AI call is made



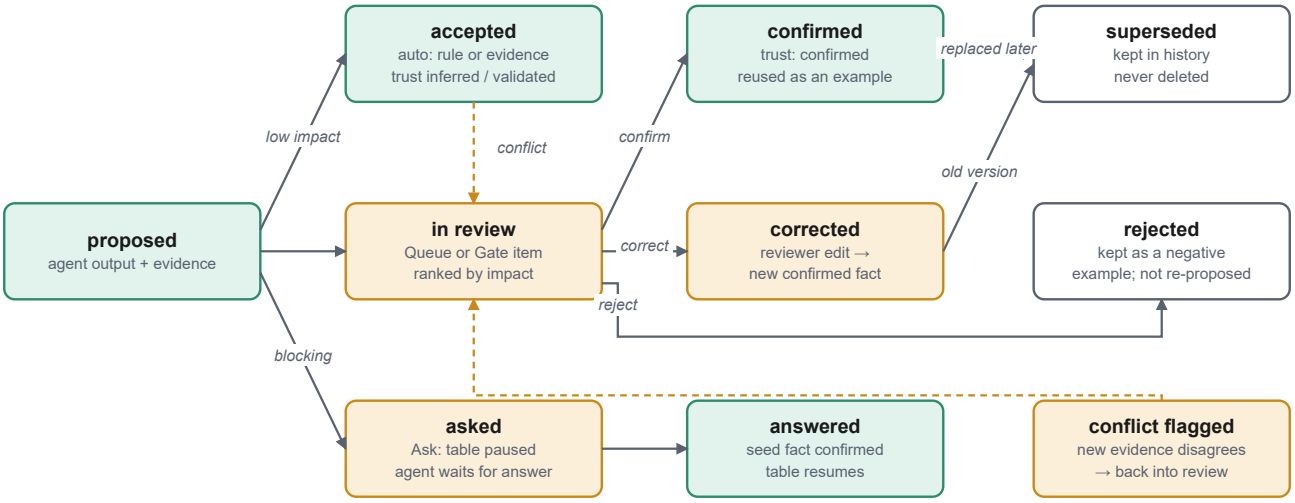
Replay mode (tests, CI, demos) answers every call from the replay store: no network, no token spend.

Batch API, server-side fallbacks and task budgets are not on Vertex AI, so the gateway provides those behaviours.

Figure 7. The model-gateway path for one AI call.

Route	Model on Vertex AI	Used by	What the model sees
translate_statement	Claude Sonnet 5.5	28	Statement text, input schemas, similar confirmed translations
translate_expression	Claude Sonnet 5.5; Opus 5.5 for complex set analysis, TOTAL, Aggr	29	Expression, ExprAST, field catalog
repair_sql	Claude Opus 5.5	25	SQL, failed rules, schemas
diagnose_mismatch	Claude Opus 5.5	26	SQL, aggregate differences, lineage; never rows
parse_export	Claude Sonnet 5.5 (Opus for complex layouts)	21	Exported report pages

8.5 How review works



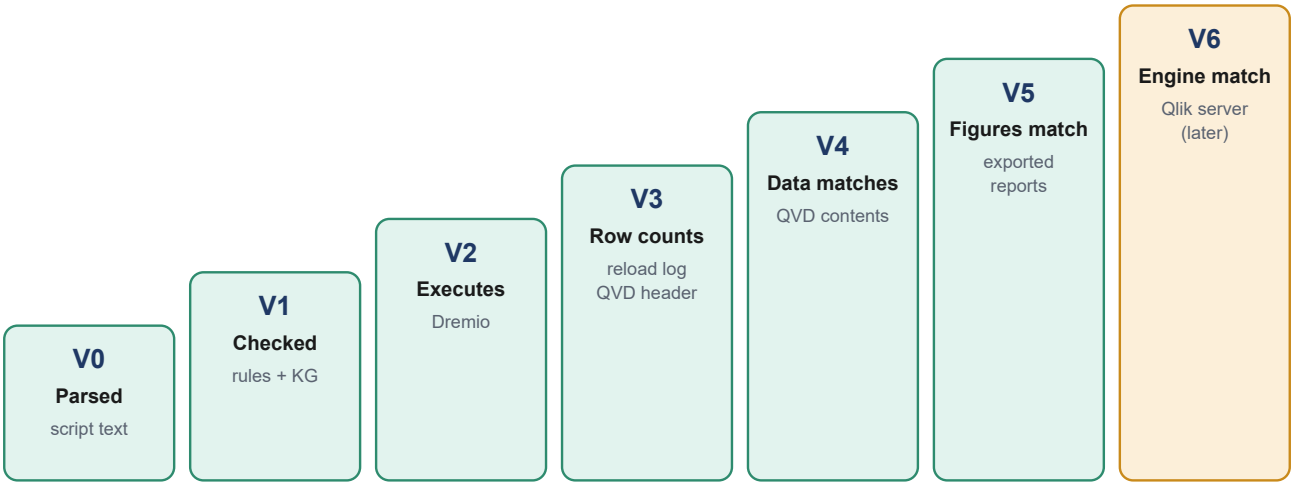
Every state change is an appended fact; nothing is overwritten. The fact log keeps both valid time and record time.

Figure 8. The life of a proposed fact.

Mode	Blocks?	Starting-stage examples
Ask	Only the affected table	A variable with no value in the script or logs
Queue	No	Uncertain source mapping; untranslated statement; join conflict

Mode	Blocks?	Starting-stage examples
Gate	No (trust stays proposed until approved)	Application scope; final SQL approval
Seed	No	Connection mapping and known variable values supplied up front
Correct	No	Reviewer edits a translation; the edit becomes a confirmed fact

9. Validation and mismatch diagnosis



Each table and figure is labelled with the highest level it reaches. Higher levels need more evidence.

Figure 9. The validation ladder. Each table and figure is labelled with the highest level it reaches.

Point-in-time alignment. Qlik evidence reflects the data at the last reload; Dremio returns current data. On Iceberg tables, queries run AT TIMESTAMP of the reload and comparisons are exact. Otherwise differences are reported with a stated tolerance and the time gap as the reason, and the result is never labelled exact.

Comparisons run in Dremio. Totals, distinct counts and per-dimension figures are computed by pushdown queries; only aggregates leave Dremio. QVD contents, when needed for V4, are read in process with DuckDB and Polars.

Symptom	Likely cause	Automatic check	Routed to
More rows than Qlik loaded	Join fan-out; duplicate keys in a copy	Key uniqueness; join cardinality per step	Join planner, checker (re-plan)
Fewer rows than Qlik loaded	Filter or variable resolved differently; NULL handling	Per-statement counts against the reload log	Extractor (variables) or Ask
Counts match, totals differ	Expression semantics: types, rounding, dual values	Column aggregates per field	Expression translator (re-translate)
Differences only in recent rows	Data changed after the reload	Time travel; comparison by date bucket	Tolerance with reason
Values from another dataset	Wrong copy or wrong connection mapping	Equivalence on candidate copies	Review queue
Table matches, figure differs	Set analysis, selection state, TOTAL or Aggr scope	Rule checks; per-dimension difference	Expression translator; diagnose route

10. Re-runs and change handling

Work is never repeated without a reason. Each change re-runs only what depends on it, found through the facts and their evidence links.

Change	What re-runs	What is kept
New version of a script	Extract, then only tables whose statements changed (statement hashes)	Facts for unchanged tables
Reviewer confirms a mapping	Resolve onwards for tables reading that source	Everything else
A Dremio view definition changes	Catalog, then resolve to reconcile for dependent tables	Independent tables
New reload log or QVD file	Reconcile only	Translations and SQL
New prompt or model version	AI steps whose memo keys change; gated by the gold-set evaluation in CI	Deterministic steps and unaffected routes
A reviewer corrects a translation	Compose, check and reconcile for that table	Other tables

11. Agents in the starting stage

Seventeen agents plus the orchestrator. Numbers match the full ADCE agent catalog of 32 agents; "lite" agents run a reduced version now and grow in later stages. Kinds: D deterministic, H hybrid.

#	Agent	Kind	Service	Role in the starting stage	AI route
1	Intake router	D	ingestion	Classify and store Qlik files per application	—
2	Schema parser	D	ingestion	Columns, types, view SQL lineage from Dremio	—
4	Entity linker (lite)	H	ingestion	Match Qlik sources to Dremio datasets	—
5	Scope registrar	D	ingestion	Register applications, spaces and datasets	—
9	Key discoverer (lite)	D	support	Key uniqueness in Dremio	—
19	Lineage (lite)	D	support	Dataset → QVD → table → visual lineage	—
20	Publisher (lite)	D	support	Write migration packs and exports	—
21	Report parser	H	reversesql	Read exported reports (Excel, PDF)	parse_export
22	Metric mapper	H	reversesql	Map measures to fields and tables	—
23	Join planner	H	reversesql	Plan joins from Qlik associations and keys	—
24	Synthesizer	H	reversesql	Compose CTE chains per table and visual	—
25	Query checker	H	reversesql	Dialect, grain, fan-out and BI semantics rules	repair_sql
26	Reconciler	H	reversesql	Execute, compare, diagnose, assign level	diagnose_mismatch

#	Agent	Kind	Service	Role in the starting stage	AI route
27	BI extractor	D	bi-adapters	Scripts and files into the BI IR	—
28	Script translator	H	bi-adapters	Load statements into SQL	translate_statement
29	Expression translator	H	bi-adapters	Measures into SQL via ExprAST	translate_expression
30	Equivalence (lite)	H	support	Detect copies of datasets Qlik reads	—
—	Orchestrator	—	orchestrator	Run the qlik_pipeline graph	—

12. Technology stack and alternatives

Each choice below is the default for the build. The alternatives were considered and remain possible: the ports and contracts (kg_port, data_port, the model-gateway adapter, the S3 API) keep most choices replaceable without touching the agents. The last column says when a choice should be revisited.

12.1 Languages and foundations

Need	Choice	Alternatives considered	Why this choice	Revisit if
Backend language	Python 3.12	TypeScript (Node); Java or Kotlin (closer to Calcite, which Dremio uses); Go	Strongest data and AI ecosystem: sqlglot, Arrow/ADBC, DuckDB, Polars, LangGraph, the Anthropic SDK; the existing knowledge engine is Python	A hot path needs native speed (a Rust extension, as qvdrs shows)
Contracts	Pydantic v2 + JSON Schema in adce-core	Protobuf/gRPC; Avro; plain dataclasses	Validation and schema export in one; native in FastAPI and FastMCP; the same models define AI output schemas	Non-Python consumers need binary contracts
Tooling	uv, ruff, pyright, pytest; pnpm, ESLint, Playwright	Poetry or pip-tools; mypy; black + flake8	Fast, monorepo-friendly, one tool per job	—

12.2 Agents and AI

Need	Choice	Alternatives considered	Why this choice	Revisit if
Orchestration	LangGraph 1.x + Postgres checkpointer	Temporal; Google ADK (Agent Development Kit); CrewAI; Microsoft Agent Framework; Prefect or Dagster; a hand-written state machine	Explicit typed graph, checkpoints, interrupt() for Ask, Send for per-table fan-out, model-neutral	Runs need durable timers over days or cross-service transactions: put Temporal around the graph
Model provider	Claude on Google Vertex AI (Opus 5.5, Sonnet 5.5, Haiku 4.5)	Gemini models on Vertex AI; Anthropic API directly; Amazon Bedrock; self-hosted open models (vLLM)	The Google requirement; strong code and SQL reasoning; structured outputs, prompt caching and PDF input are available on Vertex	A client mandates another provider, or an install is air-gapped: add an adapter, agents unchanged

Need	Choice	Alternatives considered	Why this choice	Revisit if
Model access layer	Own model-gateway service	LiteLLM proxy; Portkey; calling the SDK from each agent	Memoization for reproducible packs, record/replay, the privacy guard and budgets are product requirements that generic proxies do not cover together	A proxy can sit underneath the gateway for provider routing
Tool protocol	MCP via FastMCP 3.x	Official MCP Python SDK (lower level); REST or gRPC; framework-specific tools	One typed tool surface for internal agents and, later, external AI clients	—
Evaluation	Gold set in pytest + Langfuse datasets	promptfoo; DeepEval; Ragas; LangSmith	Self-hostable; the same tool as tracing	—

12.3 Knowledge and storage

Need	Choice	Alternatives considered	Why this choice	Revisit if
Fact log	Postgres 16	EventStoreDB (Kurrent); Kafka as the log; graph-only storage	Transactions with the outbox; bi-temporal queries in SQL; shared with review items and checkpoints; managed as Cloud SQL	Volume outgrows one database: partition first
Graph	Neo4j 5.26 LTS (projection)	Memgraph; Apache AGE; FalkorDB; Spanner Graph; Amazon Neptune (AWS only); Kuzu (archived in 2025, ruled out)	The existing engine uses it; Cypher and APOC; mature tooling; long-term support line	Edition licensing for client installs (Q-19): swap behind kg_port
Vector search (light use now)	pgvector	Neo4j vector index; Qdrant; Vertex AI Vector Search	No extra service	Large-scale semantic search in later stages
Object storage	MinIO locally, GCS in the cloud (S3 API)	Local file system; SeaweedFS; Garage	One S3 code path everywhere	MinIO community edition terms changed during 2025: confirm before shipping; swap through the S3 API
Local compute	DuckDB 1.5 + Polars	pandas; Apache DataFusion; Spark	In process, fast, Arrow-native; large data stays in Dremio	—

12.4 Data access and SQL

Need	Choice	Alternatives considered	Why this choice	Revisit if
Dremio queries	ADBC Flight SQL driver	pyarrow.flight with Dremio's client; Arrow Flight SQL JDBC or ODBC; REST SQL API with job polling	Arrow-native columnar results; a standard driver	—
Dremio metadata	REST catalog API v3; sys.jobs_recent	Querying INFORMATION_SCHEMA only	View SQL, wikis and tags are only in the catalog API	—

Need	Choice	Alternatives considered	Why this choice	Revisit if
SQL parsing and transpiling	sqlglot (dremio dialect)	Apache Calcite (JVM); sqlparse (tokenizer only); JSqlParser (JVM); SQLFluff's parser	Python; AST, transpiling across dialects, optimizer and column lineage in one library	Dialect gaps: contribute upstream, validate with Dremio EXPLAIN
Point-in-time queries	Iceberg AT TIMESTAMP / AT SNAPSHOT	Reflections or copies taken at reload time	No extra data movement	Tables are not Iceberg (Q-36): stated tolerances

12.5 Qlik processing

Need	Choice	Alternatives considered	Why this choice	Revisit if
Script parser	Lark grammar + statement splitter	ANTLR4; tree-sitter; parsimonious (PEG); a hand-written parser; open-source Qlik parsers as references	Python-native; the grammar can grow quickly from real scripts; good error positions	Performance or error recovery limits: move the grammar to ANTLR4
QVD files	Own header reader (XML) + PyQvd / qvdrs for contents	Converting through Qlik; commercial converters	Headers are plain XML; contents read only when V4 needs them	—
QlikView project folders	lxml	ElementTree	Speed and XPath	—
Qlik Engine (later)	Engine API over WebSocket (JSON-RPC) with a typed client	enigma.js (Node); qlik-cli (Qlik Cloud only); REST APIs	Stays in Python	—

12.6 Interfaces and identity

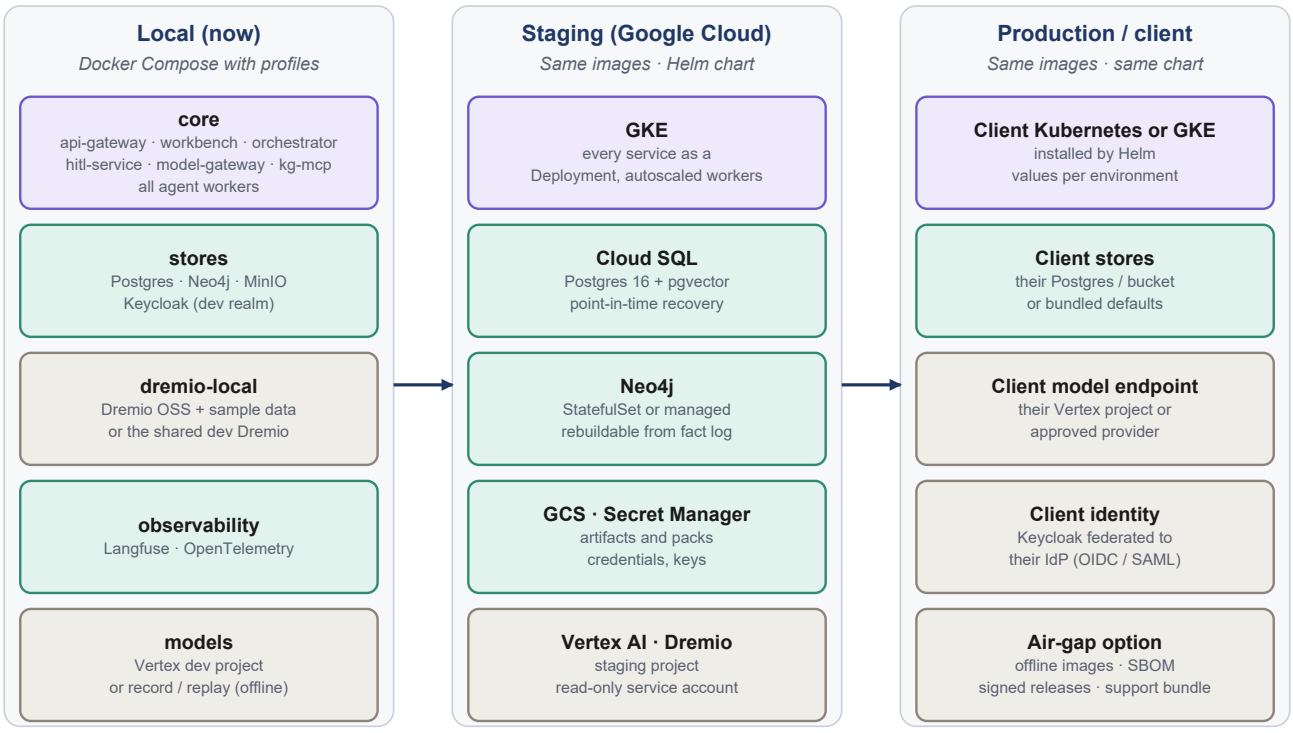
Need	Choice	Alternatives considered	Why this choice	Revisit if
Workbench	Next.js + React Flow	Vite + React SPA; Streamlit or Gradio (internal only); Retool	A production UI shipped to clients; React Flow suits lineage graphs	—
API	FastAPI	Litestar; Django REST Framework	Pydantic-native, async, OpenAPI	—
Identity	Keycloak	Google Identity Platform or IAP; Auth0 or Okta; Zitadel; Authentik	Self-hostable inside client installs; federates to client identity providers	A client supplies its own OIDC provider: configuration only

12.7 Operations

Need	Choice	Alternatives considered	Why this choice	Revisit if
Events	Postgres outbox + LISTEN/NOTIFY (Redis Streams later)	Google Pub/Sub; Kafka or Redpanda; NATS JetStream	No new infrastructure at the start; transactional with the facts	More consumers or throughput: Redis Streams (planned), or Pub/Sub in Google-only installs
Observability	Langfuse v3 + OpenTelemetry → Grafana / Prometheus	LangSmith; Arize Phoenix; Google Cloud Trace and Monitoring; Datadog	Self-hostable; understands prompts, tokens and cost	A client standard exists: export OpenTelemetry to it

Need	Choice	Alternatives considered	Why this choice	Revisit if
Deployment	Docker Compose locally; Helm on GKE for staging, production and clients	Cloud Run; VMs; Kustomize	One chart for every install; stateful services fit Kubernetes	Small installs: a Compose bundle
Infrastructure as code	Terraform	OpenTofu; Pulumi; scripts	Standard on Google Cloud	Licence preference: OpenTofu is compatible
CI/CD	GitHub Actions	Cloud Build; GitLab CI	The repository will live on GitHub	—
Secrets	Secret Manager; .env files locally	HashiCorp Vault; SOPS	Native on Google Cloud	Client installs: their secret store through the CSI driver
Supply chain	Signed images + SBOM	Unsigned images	Required by client security reviews	—

13. Environments and deployment



One build promoted unchanged: images signed, configuration only through Helm values and secrets.

Figure 10. One build promoted from local to staging to production and client environments.

Environment	Purpose	Data	Models
Local	Daily development and tests	Dremio OSS with sample data, or the shared dev Dremio	Vertex dev project, or record/replay offline
Staging (Google Cloud)	Integration, performance, demos	Read-only access to the real Dremio (Q-05, Q-06)	Vertex staging project
Production / client	Real use, inside our or the client's cloud	Client Dremio, read-only	Client's Vertex project or approved endpoint

- **Configuration only through Helm values and secrets**; no environment-specific code.
- **Database migrations** with Alembic (Postgres) and versioned Cypher migrations (Neo4j); the graph can also be rebuilt from the fact log.
- **Releases** are semantically versioned, signed and shipped with an SBOM and a support bundle command for diagnostics.

14. Security and privacy controls

Control	How it is enforced
Read-only Dremio	Read-only service account; data_port rejects anything but SELECT (sqlglot statement check)
No changes to Qlik	Only files are read; no Qlik server credentials in this stage
No row data to AI models	model-gateway privacy guard rejects row-shaped payloads on every starting-stage route; models see scripts, schemas, view SQL and aggregates only
Controlled Dremio load	Row limits, timeouts, concurrency slots, labelled queries, cached results, off-peak window for heavy checks
Least privilege	Keycloak roles for people; one service account per worker; kg-mcp policy on which agent writes which fact kinds
Audit trail	Every fact, decision, prompt version, model version and query is recorded with time and actor
Secrets	Secret Manager or the client's store; nothing in images or code
Data residency	Approved Google Cloud project and region; client installs use the client's own endpoint
Later stages	Microsoft Presidio masking before any sample values reach a model

15. Delivery plan

Phase	Weeks	What is built	Architecture switched on	Done when
Foundations	1–2	Local environment with Dremio and sample data; contracts frozen; fact log; model gateway; first parser on every sample application; reload log and QVD header readers	adce-core, kg-mcp on the fact log, data_port, model-gateway (replay), orchestrator skeleton	Sample applications parsed with measured coverage; a first Qlik table runs as SQL in Dremio
Tables	3–5	Full extraction and translation; source resolution with QVD chains; checks; V3/V4 validation; review workbench	All 4 worker services, hitl-service, workbench (2 screens)	≥ 90% statements translated; ≥ 80% tables at V3 or higher
Figures and lineage	6–8	Measure translation; export parsing; SQL per visual; lineage; figure validation	Neo4j projection, lineage view, parse_export route	≥ 3 real figures match exported reports (V5)
Harden and package	9–12	More applications; complex constructs; migration pack; load tuning; staging	Helm chart on GKE, Langfuse evaluations, signed images	Targets in section 16 met; a pack accepted by a report owner; same build in staging

Progress is reviewed at the end of each phase against these criteria. From week 6 the wider platform (profiling, dataset matching and golden records, data dictionary, estate analysis) starts building on the

same foundations.

16. Success measures

Measure	Target by week 12
Script statements translated without being flagged	At least 90% per application
Tables with matching row counts (V3 or higher)	At least 80%
Tables with matching data where QVD files exist (V4)	At least 70%
Report figures matching exported reports (V5), standard constructs	At least 70%
Results shown with their validation level	100%
Unresolved variables or sources silently assumed	None
Migration packs regenerated identically from the same inputs	100%
Cost per application translated	Measured and reported

17. Risks and mitigations

Risk	Architectural mitigation
Unusual Qlik syntax	Grammar grows from real scripts; unknown statements kept as kind: other with spans; AI route for the tail; coverage measured from week 1
Only scripts, no measure definitions	Table-level SQL and lineage still delivered; measures added when project folders or app files arrive
Data changed since the reload	Time travel on Iceberg tables; otherwise stated tolerances
Qlik reads a copy, or sources outside Dremio	Reviewed connection mapping; equivalence finds copies; the SQL reproduces exactly what Qlik reads
Missing producer in a QVD chain	Table marked not resolved; producer named in the pack
Load on Dremio	Bounded, cached, labelled, rate-limited queries; off-peak window
AI output errors	Structured outputs, re-parsing, rule checks, validation ladder, review gate
Model or provider change	Gateway adapters, versioned prompts, gold-set evaluation in CI
Knowledge engine changes	kg_port and an acceptance suite; the graph is rebuildable from the fact log

18. Open questions that shape the build

Question	Why it matters to the architecture	Default until answered
Which Qlik inputs we receive (Q-07, Q-33, Q-34)	Decides how far validation can go (V3, V4, V5)	Scripts and reload logs; QVD headers if available
Qlik data connections (Q-35)	Source resolution depends on the mapping	Mapping reviewed in week 1
Where the queries between Dremio and Qlik live (Q-37)	Lineage and translation of embedded SQL vs Dremio views	Both handled
Iceberg in Dremio (Q-36)	Exact or tolerance-based comparisons	Tolerances with reasons
Dremio edition, access and permissions (Q-05, Q-06)	data_port features and local setup	Dremio OSS locally with sample data

Question	Why it matters to the architecture	Default until answered
What "Google API" means for models (Q-03)	Default model routes	Claude on Vertex AI
Google Cloud setup (Q-04)	Projects, regions, quotas	Dev and staging projects
Neo4j edition (Q-19)	Licensing in client installs	Community for development

The full list with owners and status is kept in docs/open-questions.md.

19. Decision requested

> **Approve the 12-week starting stage on this architecture: Qlik-to-SQL reconstruction on 2–3 real Qlik applications, with read-only Dremio access, the Qlik artifacts in section 18, a Google Cloud project for model access, and an infrastructure and model budget sized on Google Cloud pricing in the first two weeks.**